Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport

Lucrarea de laborator Nr. 6

Disciplina: Tehnici de programare

Tema: Informații invizibile: forensics și steganografie

A efectuat: st. gr. SI-264
Radu Sologub

A verificat: **Reițman Patricia**, asis. univ.

Chișinău, 2026

Lucrarea de laborator nr. 6

Introducere — sarcina şi mediul de lucru

Lucrarea are două părţi. În Lucrarea A am construit un decoder Python pentru URL, hexazecimal şi Base64, straturi multiple, XOR cu o cheie de un octet şi recuperarea parolelor prin dicţionar. În Lucrarea B am identificat tipurile reale ale fişierelor, am extras text imprimabil şi metadate EXIF/GPS, am separat o arhivă ZIP ascunsă la sfârşitul unei imagini şi am ascuns apoi recuperat un mesaj prin LSB.

Am folosit numai fişierele create local de generatorul primit la laborator. Fotografia şi metadatele sale sunt sintetice: valorile Apple, iPhone 13, data şi coordonatele provin din generator, nu dintr-o fotografie personală. Dovezile sunt codul executabil şi output-ul terminalului.

Execuţia documentată a avut loc la 13 octombrie 2026, pe Linux, cu Python 3.14.7, Pillow 12.3.0 şi pytest 9.1.1. Pregătirea efectuată a fost:

```
python -m venv --system-site-packages .venv
.venv/bin/python -m pip install Pillow pytest
.venv/bin/python probe/pregateste.py
```

Generatorul a produs mesajul în straturi, fişierul XOR, lista de hash-uri şi dicţionarul, imaginea JPEG cu EXIF, imaginea PNG cu ZIP ataşat şi fişierele provocării opţionale. Codul generatorului a fost păstrat fără modificări. Comenzile de mai jos se rulează din directorul lucrării.

1. Text, octeţi şi recunoaşterea codării — A1–A3

Textul se converteşte în octeţi cu `encode()`. Indexarea unui obiect bytes întoarce un număr: primul octet al lui Salut este 83. Reprezentarea hexazecimală poate fi convertită înapoi fără a schimba mesajul.

```
$ .venv/bin/python -c 's = "Salut"; b = s.encode(); print(b, b[0]); print(b.hex());
print(bytes.fromhex(b.hex()))'
b'Salut' 83
53616c7574
b'Salut'
```

Funcţia `ghici_strat()` verifică mai întâi secvenţele URL de forma `%xx`, apoi un şir hexazecimal de lungime pară şi apoi alfabetul Base64. `desfa()` întoarce tipul şi rezultatul ca bytes, folosind `unquote()`, `bytes.fromhex()` sau `base64.b64decode(..., validate=True)`.

Demonstraţia URL a produs calea /etc, demonstraţia hexazecimală a produs Hello, iar Base64 a produs Salut. Rezultatele observate sunt:

```
$ .venv/bin/python arsenal/forensics/decode.py %2Fetc --fisa rezultate/fisa_url.json
{
  "strat": "url",
  "rezultat": "/etc",
  "straturi": [
    "url"
  ]
}
```

```
$ .venv/bin/python arsenal/forensics/decode.py 48656c6c6f --fisa rezultate/fisa_hex.json
{
  "strat": "hex",
  "rezultat": "Hello",
  "straturi": [
    "hex"
  ]
}

$ .venv/bin/python arsenal/forensics/decode.py U2FsdXQ= --fisa rezultate/fisa_base64.json
{
  "strat": "base64",
  "rezultat": "Salut",
  "straturi": [
    "base64"
  ]
}
```

Recunoașterea după alfabet este o euristică. De exemplu, /etc conține numai caractere acceptate și de Base64. Decoderul păstrează rezultatul URL citibil dacă interpretarea ulterioară nu produce text UTF-8 sau un candidat XOR imprimabil. Nu presupune că orice potrivire de alfabet reprezintă încă un strat real.

2. Straturi multiple, XOR și dicționar — A4–A6

2.1. Desfacerea tuturor straturilor

Un rezultat intermediar imprimabil poate fi tot o codare. Mesajul probei trece prin Base64, încă un Base64 și apoi hexazecimal. De aceea, bucla nu se oprește doar pentru că un strat intermediar este text ASCII. Ea continuă cât timp mai poate desface un strat și permite cel mult opt straturi.

```
$ .venv/bin/python arsenal/forensics/decode.py probe/mesaj.txt --fisier
{
  "strat": "hex",
  "rezultat": "FLAG{straturi_de_encoding}",
  "straturi": [
    "base64",
    "base64",
    "hex"
  ]
}
```

Mesajul recuperat este `FLAG{straturi_de_encoding}`. Fișa JSON conține rezultatul, ultimul tip de strat și lista completă a transformărilor.

2.2. XOR cu o cheie de un octet

Funcția `sparge_xor()` încearcă valorile 0–255. Pentru fiecare cheie aplică XOR fiecărui octet și verifică dacă rezultatul este în intervalul ASCII imprimabil 32–126. Pentru proba primită, primul candidat acceptat are cheia 128, adică 0x80.

```
$ .venv/bin/python arsenal/forensics/decode.py probe/xor.bin --fisier --fisa
rezultate/fisa_xor.json
{
  "strat": "xor",
  "rezultat": "Un singur octet tine tot secretul... gaseste-l! FLAG{xor_un_octet}",
  "straturi": [
    "xor"
  ],
  "cheie": 128
}
```

Textul include mesajul `FLAG{xor_un_octet}`. Acest filtru este suficient pentru proba generată; caracterul imprimabil al unui rezultat nu demonstrează singur că orice mesaj XOR arbitrar a fost identificat fără ambiguitate.

2.3. Recuperarea parolelor prin dicționar

Algoritmul este identificat din lungimea unei reprezentări hexazecimale valide: 32 de caractere pentru MD5, 40 pentru SHA-1 și 64 pentru SHA-256. Pentru fiecare cuvânt din dicționar calculez valoarea cu algoritmul respectiv și o compar cu ținta. Operația este o căutare prin dicționar, nu decriptarea unui hash.

Am verificat toate cele cinci ținte MD5 generate, iar parolele găsite sunt password, dragon, qwerty, letmein și monkey:

```
$ .venv/bin/python arsenal/forensics/decode.py probe/hashuri.txt --fisier --hashuri --fisa
rezultate/fisa_hashuri.json
[
  {
    "strat": "md5",
    "rezultat": "password"
  },
  {
    "strat": "md5",
    "rezultat": "dragon"
  },
  {
    "strat": "md5",
    "rezultat": "qwerty"
  },
  {
    "strat": "md5",
    "rezultat": "letmein"
  },
  {
    "strat": "md5",
    "rezultat": "monkey"
  }
]
```

3. Interfața de comandă, fișa JSON și testele — A7

`decode.py` primește șirul ca argument sau citește octeții unui fișier cu `--fisier`. `--hashuri` procesează câte o țintă pe linie. `--wordlist` permite alegerea dicționarului, iar `--fisa` separă rezultatele demonstrațiilor. Fără această ultimă opțiune, unealta scrie fișa cerută în `arsenal/forensics/fisa.json`.

Testul Base64 cerut în fișă este păstrat în `teste/test_decode.py`:

```
from arsenal.forensics.decode import desfa

def test_base64():
    strat, b = desfa("U2FsdXQ=")
    assert strat == "base64"
    assert b == b"Salut"
```

Suita verifică și ordinea recunoașterii, mesajul cu trei straturi, limita de opt straturi, proba XOR, dicționarele pentru trei algoritmi, coordonatele GPS, arhiva extrasă și recuperarea LSB. Cazurile nevalide acoperă Base64 incorect, intrarea XOR goală, hash-ul cu formă incorectă, lipsa arhivei, capacitatea insuficientă a imaginii și lipsa terminatorului.

Comanda executată a fost `.venv/bin/python -m pytest -v`. Rezultatul final: **34 de teste trecute, fără teste eșuate**. În plus, toate comenzile CLI prezentate în raport au fost executate până la capăt.

4. Tipuri reale, strings, EXIF/GPS și carving — B1–B4

4.1. Semnătura și textul imprimabil

`tip_real()` citește primii opt octeți în mod binar. Recunoaște PDF, PNG, JPEG și ZIP/DOCX după semnătură, indiferent de extensie. Proba foto.jpg este JPEG, ascuns.png rămâne PNG chiar dacă are o arhivă la coadă, iar arhiva separată este ZIP.

`extrage_strings()` caută secvențe ASCII imprimabile de cel puțin patru caractere cu expresia regulată pe bytes `rb"[ -~]{4,}"`. Numele secret.txt și mesajul ascuns apar printre șirurile recuperate.

4.2. Metadatele și coordonatele

Imaginea conține Make = Apple, Model = iPhone 13 și DateTime = 2026:09:14 10:30:00. Conversia coordonatelor folosește formula:

```
grade zecimale = grade + minute / 60 + secunde / 3600
```

Semnul se inversează pentru emisferele S și W. Din datele GPS generate am obținut latitudinea **47.01666667** și longitudinea **28.85750000**, corespunzătoare punctului indicat în fișă pentru Chișinău.

4.3. Separarea arhivei

Semnătura ZIP `PK\x03\x04` a fost găsită la octetul **782** al imaginii ascuns.png. Scriptul salvează partea de la acea poziție în gasit.zip și extrage secret.txt. Conținutul său este `FLAG{ascuns_prin_carving}`.

Output-ul complet al verificărilor B1–B4 este:

```
$ .venv/bin/python forensics.py
B1, foto.jpg: jpeg
B1, ascuns.png: png
B2, șiruri imprimabile din ascuns.png:
IHDR
IDATx
IEND
N~M]
secret.txtFLAG{ascuns_prin_carving}
N~M]
secret.txtPK
B3, metadate EXIF:
Model = iPhone 13
DateTime = 2026:09:14 10:30:00
Make = Apple
Latitudine: 47.01666667
Longitudine: 28.85750000
B4, arhiva extrasă de la octetul 782
Fișier extras: secret.txt
FLAG{ascuns_prin_carving}
```

5. Ascunderea și dezvăluirea prin LSB — B5–B6

`stego.py` convertește gazda în RGB și folosește canalele în ordinea R, G, B pentru fiecare pixel. Fiecare bit al mesajului înlocuiește numai ultimul bit al canalului, prin `(canal & ~1) | bit`. Octeții sunt înscriși de la bitul cel mai semnificativ la cel mai puțin semnificativ.

Mesajul este codificat în UTF-8 și urmat de un octet zero. Acest terminator este necesar pentru metoda de extragere din fișă: fără el, citirea ar continua în pixelii rămași după mesaj. Programul verifică dacă mesajul împreună cu terminatorul încape și cere salvare PNG, fără recomprimarea cu pierdere specifică JPEG.

Ascunderea a înscris 184 de biți, inclusiv terminatorul:

```
$ .venv/bin/python stego.py ascunde probe/foto.jpg 'FLAG{ascuns_in_pixeli}' stego.png
Mesaj ascuns: FLAG{ascuns_in_pixeli}
Biți înscriși, inclusiv terminatorul: 184
Imagine salvată: stego.png
```

La extragere se adună ultimii biți în aceeași ordine, se formează octeți de câte opt biți și se oprește la primul octet zero. Mesajul recuperat este identic cu cel ascuns:

```
$ .venv/bin/python stego.py dezvaluie stego.png
Mesaj dezvăluit: FLAG{ascuns_in_pixeli}
```

Imaginea rezultată este PNG, 640 × 480. Compararea canalelor RGB ale gazdei decodate cu rezultatul a identificat **88 de canale modificate**, fiecare cu o diferență maximă de **1**. Biții superiori au rămas neschimbați, iar canalele de după cei 184 de biți înscriși sunt identice. Verificarea confirmă atât recuperarea mesajului, cât și modificarea exclusivă a LSB.

6. Provocarea opțională

Fișierul challenge.bin conține Base64 peste un mesaj XOR. Decoderul a desfăcut Base64, a recuperat cheia 128 și a obținut `FLAG{lant_complet_despicat}`. Verificarea din fișierul-indiciu primit a reușit.

```
$ .venv/bin/python arsenal/forensics/decode.py probe/challenge.bin --fisier --verifica
probe/challenge_hint.txt --fisa rezultate/fisa_challenge.json
{
  "strat": "xor",
  "rezultat": "FLAG{lant_complet_despicat}",
  "straturi": [
    "base64",
    "xor"
  ],
  "cheie": 128,
  "verificare_md5": true
}
```

Concluzii

Am construit și executat un decoder pentru cele trei codări cerute, straturi multiple, XOR și căutare prin dicționar. Am recuperat mesajul în trei straturi și toate parolele din probe. Funcțiile de forensics au identificat semnăturile reale, au expus textul și metadatele GPS și au separat arhiva atașată unei imagini. Mesajul LSB a fost recuperat identic după salvare și redeschidere.

Lucrarea arată că extensia unui fișier și aspectul unei imagini nu descriu tot conținutul lor. Codarea, hash-urile și steganografia sunt mecanisme diferite, iar recunoașterea euristică trebuie confirmată prin rezultatul obținut. Cele 34 de teste și execuțiile documentate verifică funcționarea pe probele laboratorului și cazurile-limită relevante.

Bibliografie

1. UTM, FCIM. *Laboratorul 6. Informații invizibile: forensics și steganografie*. Fișa disciplinei Tehnici de programare, 2026, și generatorul de probe furnizat pentru lucrare.
2. Python Software Foundation. base64 — Base16, Base32, Base64, Base85 Data Encodings.
3. Python Software Foundation. hashlib — Secure hashes and message digests.
4. Python Software Foundation. urllib.parse — URL parsing utilities.
5. Python Software Foundation. zipfile — Work with ZIP archives.
6. Pillow. Image module.